

Draft: Type-based escape analysis with unions

Keywords: type systems, escape analysis, subtyping.

Abstract. Programming languages frequently utilize escape analysis techniques to ensure the safety of compiler optimizations and language features such as borrowing and effect systems. To achieve higher precision, particularly in interprocedural contexts, this analysis is often implemented on the type level. However, type-level escape analyses tend to be rigid and may impose excessive annotation requirements on programmers.

In this paper, we introduce a novel type-based escape analysis approach that incorporates a minimum (union) operation on lifetimes, enabling user-provided type annotations accurately capture a function’s data flow, thereby making them simpler and more intuitive to write. Additionally, our approach simplifies lifetime tracking for data types through the use of existential lifetimes. Finally, we provide a prototype implementation of the corresponding type-checking algorithm.

1 Introduction

Escape analysis is a fundamental technique used by optimizing compilers. Specifically, when an object does not escape the scope in which it is allocated, it can be safely placed on the stack instead of the heap, thereby reducing garbage collection overhead and improving overall program performance [2]. Moreover, escape analysis plays a crucial role in verifying the safety of various language features. For instance, if a first-class function does not leak beyond its intended scope, non-local returns are guaranteed to be safe [1]. In addition, ownership-based resource management mechanisms often utilize borrowing, permitting the use of resources without requiring ownership transfer within confined scopes [7, 6]. Furthermore, escape analysis can be leveraged to enforce safety in contextual polymorphism within capability-based effect systems [5, 3].

Despite its usefulness and interprocedural abilities, languages often avoid using type-based escape analysis, preferring classical approaches based on monotone framework [2]. This preference stems primarily from the inherent rigidity or complexity of the resulting type systems. For example, the Rust programming language’s lifetime system necessitates support for variance inference, lifetime polymorphism for compound data types, and lifetime constraints [7]. Together, these requirements contribute to a flexible but challenging model both for implementers and users.

In this work, we present a simplified type-based escape analysis that intentionally trades off some expressivity to simplify user-provided type annotations. Specifically, we assume lifetimes of data types are always covariant, while still preserving safety guarantees. Additionally, we introduce existential lifetimes to automate the handling of data type lifetimes. Finally, our approach incorporates union types, which make type signatures transparently describe a function’s dataflow.

The remainder of this paper is organized as follows. Section 2 provides an informal introduction to our escape analysis approach, illustrated through practical examples. In Section 3, we present a formalization of our system using a rigorous formal calculus. Section 4 discusses details of our prototype implementation. Finally, Section 5 compares our design with existing approaches.

2 Solution overview

In the following, we provide an informal overview of the type-based escape analysis technique introduced in this paper.

41 We refer to values that must not escape a given scope as *tracked*, and likewise, their types are designated
42 as tracked. To differentiate between tracked and non-tracked types, each type T is annotated with a special
43 label `lab`, referred to as a lifetime:

```
44 T'lab /* or for function types */ (A)'lab -> B
```

45 The `free` lifetime label designates non-tracked values. In contrast, the `local` label appears in tracked
46 types and is treated specially by the type system to prevent their escape. Namely, tracked values also
47 cannot be returned from functions. Thus, we can specify in type of a higher-order function that it do not
48 leak an argument function. For example, the eager `map` function calls its argument `inplace`:¹²

```
49 fun map(xs: List<a>, f: (a)'local -> b): List<b>
```

50 To enable a function to propagate its arguments through its result, polymorphism over lifetime labels
51 can be used. In essence, this is an explicit specification of the function's dataflow directly in the type
52 signature. For instance, the lazy `map` function does not eagerly compute and return the final result;
53 instead, it immediately yields a collection that encapsulates the processing logic. This behavior can be
54 captured by assigning the same lifetime variable `l` to both the argument function `f` and the result type.
55 Consequently, the type system records that any tracked capabilities captured by `f` may be retained within
56 the result. Thus, such a function continues to support contextual polymorphism, as it can only permit `f`
57 to escape with the result, and the call site is made aware that the result must not escape its prescribed
58 context:

```
59 fun lazyMap(xs: LazyList<a>, f: (a)'l -> b): LazyList<b>'l  
60  
61 fun printAll(file: File'local, xs: LazyList<Int>) {  
62     let ys: LazyList<Int>'local = lazyMap(xs, (x) => file.write(x))  
63     ys.collect() // force execution  
64 }
```

65 To express the capture of two or more lifetime-polymorphic values, we employ a syntax with plus that
66 denotes the minimum of their lifetimes:

```
67 fun compose(f: (b)'lf -> c, g: (a)'lg -> b): (a)'lf+lg -> c =  
68     (x) => f(g(x))
```

69 For the sake of simplicity, lifetime polymorphism is not supported for compound datatypes. Instead,
70 their lifetimes are computed as the minimum of the lifetimes of their components; the same mechanism
71 is applied to closures. Upon destructuring, the lifetimes of the individual components are, in turn,
72 approximated by the lifetime of the original compound datatype.

```
73 fun makeRepository(file: File'lf, conn: Connection'lc): Repository'lf+lc =  
74     Repository(file, conn)  
75  
76 fun registerUser(repo: Repository'local, user: User) {  
77     let conn: Connection'local = repo.conn  
78     conn.send(user)  
79 }
```

¹We adopt the Haskell naming convention, where type names beginning with a lowercase letter denote type variables, which are implicitly universally quantified.

²When we omit the explicit lifetime annotation, we assume the `free` lifetime.

Variables	x, y, z, f, g
Data constructors	K
Values	$v ::= x \mid K \mid \Lambda \bar{l} \bar{\alpha}. v \mid K \bar{\Delta} \bar{\tau} \bar{v} \mid \lambda \bar{x} : \bar{\tau}. t$
Terms	$t, u ::= v \mid t \bar{\Delta} \bar{\tau} \mid t \bar{t} \mid \text{match } t \{ \overline{K \bar{x} \rightarrow t} \}$
Evaluation context	$E ::= \square \mid E \bar{\Delta} \bar{\tau} \mid E \bar{t} \mid v (\bar{v}, E, \bar{t})$ $\mid K \bar{\Delta} \bar{\tau} (\bar{v}, E, \bar{t}) \mid \text{match } E \{ \overline{K \bar{x} \rightarrow t} \}$

Figure 1: Syntax of $Core_{\Delta}$.

80 We define subtyping so that non-tracked values may be used in positions where tracked values are
81 expected: `free` <: `local`. Moreover, for simplicity, all type constructors are covariant with respect to
82 their lifetimes. So, at the destruction site we consider stored lifetimes as covariant projections (see 3.4).

83 Following [3], we permit tracked types to instantiate polymorphic type parameters, thereby making
84 them first-class citizens in the language. To control when tracked types are appropriate for instantiation,
85 we use bounded polymorphism. For example, the `map` function uses values of type `a` inside and returns
86 values of type `b` directly to the caller. As a result, these type parameters may be instantiated with tracked
87 types:^{3 4}

```
88 fun map(xs: List<a>, f: (a)'l -> b): List<b>'l
89     where a <: Any'local, b <: Any'local
```

90 Upon capturing, an unknown lifetime of a type argument is approximated by the lifetime of its bound.
91 Consider the most permissive type for the `lazyMap` function:

```
92 fun lazyMap(xs: LazyList<a>, f: (a)'lf -> b): LazyList<b>'lf+la
93     where a <: Any'la, b <: Any'lb
```

94 3 Formalization

95 We formalize our approach as a call-by-value calculus $Core_{\Delta}$, which is basically a polymorphic λ -calculus
96 extended with data constructors, subtyping and lifetime labels.

97 3.1 Syntax and semantics of $Core_{\Delta}$

98 The term-level syntax of the $Core_{\Delta}$ is given on the Figure 1. The overline denotes a bunch of corresponding
99 entities. Type application and type abstraction have two groups of arguments: lifetimes and types.

100 The type-level syntax of $Core_{\Delta}$ is given on the Figure 2. We will use $+$ on lifetimes in prefix position
101 when taking the minimum of a bunch of lifetimes (like $+\bar{\Delta}$) and in infix position for the given lifetimes
102 (like $\Delta_1 + \Delta_2$). All monotypes are annotated with lifetimes. Type schemas contain lifetime parameters
103 and bounded type parameters.

104 The reduction semantics of $Core_{\Delta}$ is given on the Figure 3.

Type variables	α, β
Type constructors	T
Lifetime variables	l
Lifetimes	$\Delta ::= l \mid local \mid free \mid +\bar{\Delta}$
Monotypes	$\tau, \sigma, \eta ::= \alpha \mid T \Delta \bar{\tau} \mid \bar{\tau} \Delta \rightarrow \sigma$
Type schemas	$s ::= \forall \bar{l} (\alpha <: \tau). \sigma$
Typing contexts	$\Gamma ::= \emptyset \mid x : s, \Gamma \mid l <: \Delta, \Gamma \mid \alpha <: \tau, \Gamma$ $\mid K : \forall \bar{l} \bar{\alpha}. \bar{\tau} \rightarrow T \Delta \bar{\alpha}, \Gamma$

Figure 2: Syntax of types of $Core_{\Delta}$.

$$\begin{array}{lll}
(\text{tapp}) & (\bar{l} \bar{\alpha}. t) \bar{\Delta} \bar{\tau} & \longrightarrow [\bar{l} \rightarrow \bar{\Delta}, \bar{\alpha} \rightarrow \bar{\tau}] t \\
(\text{app}) & (\lambda \bar{x}. t) \bar{v} & \longrightarrow [\bar{x} \rightarrow \bar{v}] t \\
(\text{match}) & \text{match } K \Delta \bar{\tau} \bar{v} \{ \overline{K_i \bar{x}_i \rightarrow u_i} \} & \longrightarrow [\bar{x}_k \rightarrow \bar{v}] u_k, \text{ where } K = K_k \\
& \frac{t \longrightarrow t'}{E[t] \longrightarrow E[t']} (\text{step}) &
\end{array}$$

Figure 3: Operational semantics of $Core_{\Delta}$.

$$\begin{array}{c}
\boxed{\Gamma \vdash \Delta' <: \Delta} \\
\\
\frac{}{\Gamma \vdash free <: \Delta} SubFree \qquad \frac{}{\Gamma \vdash \Delta <: local} SubLocal \\
\frac{l <: \Delta \in \Gamma}{\Gamma \vdash l <: \Delta} SubCtx_{\Delta} \qquad \frac{\forall \Delta' \in \bar{\Delta}'. \exists \Delta \in \bar{\Delta}. \Gamma \vdash \Delta' <: \Delta}{\Gamma \vdash +\bar{\Delta}' <: +\bar{\Delta}} Sub+ \\
\\
\boxed{\Gamma \vdash \sigma <: \tau} \\
\\
\frac{\alpha <: \tau \in \Gamma}{\Gamma \vdash \alpha <: \tau} SubCtx \qquad \frac{\Gamma \vdash \Delta' <: \Delta}{\Gamma \vdash T \Delta' \bar{\tau} <: T \Delta \bar{\tau}} SubData \\
\\
\frac{\Gamma \vdash +lt_{\Gamma}(\tau) <: \Delta}{\Gamma \vdash \tau <: Any \Delta} SubAny \qquad \frac{\overline{\Gamma \vdash \tau <: \tau'} \quad \Gamma \vdash \Delta' <: \Delta \quad \sigma' <: \sigma}{\Gamma \vdash \bar{\tau}' \Delta' \rightarrow \sigma' <: \bar{\tau} \Delta \rightarrow \sigma} SubFun
\end{array}$$

Figure 4: Subtyping rules of $Core_{\Delta}$.

$$\boxed{\Gamma \vdash t : s}$$

$$\frac{x : s \in \Gamma}{\Gamma \vdash x : s} \textit{Var}$$

$$\frac{\overline{\alpha <: \sigma}, \Gamma \vdash t : \tau}{\Gamma \vdash \Lambda \bar{l} (\alpha <: \sigma). t : \forall \bar{l} (\alpha <: \sigma). \tau} \textit{TLam} \quad \frac{x : \forall \bar{l} (\overline{\alpha <: \tau}). \sigma \in \Gamma \quad \overline{\Gamma \vdash \tau' <: [\bar{l} \rightarrow \Delta]} \tau}{\Gamma \vdash x \Delta \bar{\tau}' : [\alpha \rightarrow \tau'] [\bar{l} \rightarrow \Delta] \sigma} \textit{TApp}$$

$$\frac{\overline{x : \tau}, \Gamma \vdash t : \sigma \quad \Gamma \vdash_{fv(t) \setminus \bar{x}} \bar{s} \quad \textit{local} \notin lt_{\emptyset}(\sigma)}{\Gamma \vdash \lambda \bar{x} : \bar{\tau}. t : \bar{\tau} \left(+lt_{\Gamma}(\bar{s}) \right) \rightarrow \sigma} \textit{Lam} \quad \frac{\Gamma \vdash t : \bar{\tau} \Delta \rightarrow \sigma \quad \overline{\Gamma \vdash u : \tau'} \quad \overline{\Gamma \vdash \tau' <: \tau}}{\Gamma \vdash t \bar{u} : \sigma} \textit{App}$$

$$\boxed{\Gamma \vdash_{\bar{x}} \bar{\tau}} \quad \frac{\overline{x : s} \subset \Gamma}{\Gamma \vdash_{\bar{x}} \bar{s}} \textit{TypesOf}$$

Figure 5: Typing basics of $Core_{\Delta}$.

3.2 Subtyping relation

The subtyping relation (Figure 4) on lifetimes forms a bounded lattice with *local* as the top and *free* as the bottom. Thus, the minimum of lifetimes is the least upper bound on this lattice. If $\Delta' <: \Delta$ we say that Δ' outlives Δ .

We use the operation $lt_{\Gamma}(\cdot)$ to collect all lifetime restrictions from a type. It is defined as the following:

$$\begin{aligned}
lt_{\Gamma}(\alpha) &= lt_{\Gamma}(\tau) \text{ if } \alpha <: \tau \in \Gamma, \emptyset \text{ otherwise} \\
lt_{\Gamma}(T \Delta \bar{\tau}) &= \overline{lt_{\Gamma}(\bar{\tau})} \cup \{\Delta\} \\
lt_{\Gamma}(\bar{\tau} \Delta \rightarrow \bar{\sigma}) &= \{\Delta\} \\
lt_{\Gamma}(\forall \bar{l} (\overline{\alpha <: \tau}). \sigma) &= lt_{\Gamma \setminus \bar{l} \setminus \bar{\alpha}}(\sigma) \setminus \bar{l}
\end{aligned}$$

The lifetime of a type variable is approximated by the lifetime of its bounding type, if such a bound exists.

In some cases, we intentionally employ an empty context to exclude the influence of bounds.

The rule *SubAny* says that a special *Any'* Δ type is a supertype of τ if all components of τ outlive Δ .

3.3 Typing basics of $Core_{\Delta}$

The *Lam* rule (Figure 5) is crucial for our escape analysis as it prevents leaking of tracked values by prohibiting types with *local* lifetime present in the return type. Also, it defines that a lifetime of a function is determined by lifetimes of all captured values.

3.4 Typing data of $Core_{\Delta}$

Similar to functions, lifetime of a data constructor is approximated with minimum of lifetimes of all the data types it contains (Figure 6). So the lifetime parameters of a data constructor do not present directly in the resulting type, which makes them existential types.

³We assume that **Any** type serves as the top element in the type lattice.

⁴To preserve backward compatibility, the type **Any'****free** may be used as the default bound.

$$\boxed{\Gamma \vdash t : s}$$

$$\frac{K : \forall \bar{l} \bar{\alpha}. \bar{\tau} \rightarrow T \left(+\overline{lt_{\bar{\theta}}(\tau)} \right) \quad \bar{\alpha} \in \Gamma}{\Gamma \vdash K : \forall \bar{l} \bar{\alpha}. \bar{\tau} \rightarrow T \left(+\overline{lt_{\bar{\theta}}(\tau)} \right) \quad \bar{\alpha}} \textit{Ctor}$$

$$\frac{\Gamma \vdash t : T \Delta \bar{\tau} \quad \Gamma \vdash K_i : \forall \bar{l}_i \bar{\alpha}. \bar{\sigma}_i \rightarrow T \left(+\overline{lt_{\bar{\theta}}(\sigma_i)} \right) \quad \bar{\alpha} \quad \overline{l'_i \text{ fresh}} \quad \overline{l'_i <: \Delta, x_i : [\bar{\alpha} \rightarrow \bar{\tau}] [l_i \rightarrow l'_i] \sigma_i, \Gamma \vdash u_i : \eta_i}}{\Gamma \vdash \textit{match } t \{ \overline{K_i \bar{x}_i \rightarrow u_i} \} : \textit{lub} \left(\overline{\textit{elim}_{\Delta, \bar{l}_i}(\eta_i)} \right)} \textit{Match}$$

Figure 6: Typing data of $Core_{\Delta}$.

121 Work with existential lifetimes requires special care in the *Match* rule. For each data constructor K_i
 122 and its existential lifetimes $\bar{l}_i$ we generate fresh lifetimes $\bar{l}'_i$ and approximate their bound with the lifetime
 123 of the whole data structure.

124 For each pattern-matching branch we need to eliminate fresh lifetimes from the resulting type, since
 125 they are not bound outside. The $\textit{elim}_{\Delta, \bar{l}}^p(\cdot)$ function eliminates lifetimes $\bar{l}$, considering the lower bound
 126 Δ , considering position p ranging over $+$, $-$ and *inv* (let $p^{-1} = +$ if $p = -$ and $-$ if $p = +$ and *inv* if $p =$
 127 *inv*):

$$\begin{aligned}
\textit{elim}_{\Delta, \bar{l}}^p(\alpha) &= \alpha \\
\textit{elim}_{\Delta, \bar{l}}^p(K \Delta' \bar{\tau}) &= K \left(\textit{elim}_{\Delta, \bar{l}}^p(\Delta') \right) \left(\overline{\textit{elim}_{\Delta, \bar{l}}^{\textit{inv}}(\tau)} \right) \\
\textit{elim}_{\Delta, \bar{l}}^p(\bar{\tau} \Delta' \rightarrow \sigma) &= \left(\overline{\textit{elim}_{\Delta, \bar{l}}^{p^{-1}}(\tau)} \right) \left(\textit{elim}_{\Delta, \bar{l}}^p(\Delta') \right) \rightarrow \textit{elim}_{\Delta, \bar{l}}^p(\sigma) \\
\textit{elim}_{\Delta, \bar{l}}^p(l') &= \begin{cases} l' & \text{if } l' \notin \bar{l} \\ \Delta & \text{if } l' \in \bar{l} \wedge p = + \\ \textit{free} & \text{if } l' \in \bar{l} \wedge p = - \\ \textit{error} & \text{if } l' \in \bar{l} \wedge p = \textit{inv} \end{cases} \\
\textit{elim}_{\Delta, \bar{l}}^p(+\overline{\Delta'}) &= + \left(\overline{\textit{elim}_{\Delta, \bar{l}}^p(\Delta')} \right) \\
\textit{elim}_{\Delta, \bar{l}}^p(\Delta') &= \Delta'
\end{aligned}$$

128 When an existential lifetime occurs in invariant position, it cannot be safely approximated to the lower
 129 or the upper bound, thus, the error is reported. Otherwise, this situation can be solved by introducing
 130 bivariate projection, but we dedicate this to the future work [9].

131 The result type of the whole pattern-matching expression is computed as the least upper bound (*lub*)
 132 of branch types with eliminated lifetimes.

133 4 Implementation

134 We provide the type-checker prototype implementation written in Haskell. The core logic of the type-
 135 checker is quite minimalistic and takes two hundred lines of code. Mostly, it straight-forwardly follows
 136 the rules described in Section 3, except that lifetimes are represented in normalized form:

137 `data Lt = LtLocal | LtMin (Set LtName)`

138 Here empty set denotes the `free` lifetime.

139 5 Related work

140 The Rust language supports type-based escape analysis to ensure memory safety when using references [7].
141 This is achieved through a combination of lifetime polymorphism, variance inference, and lifetime constraints, which together provide significant representational power but also increase the overall complexity
142 of the type system. Despite that, the minimum of lifetimes can be achieved only indirectly through an
143 additional lifetime constraints or by making all corresponding lifetimes equal. In contrast, our approach
144 provides a way to express minimum directly.

145 In modal OCaml, the `local` modality serves to prevent borrowed references from escaping their defining region, ensuring safe memory use and enabling optimizations such as stack allocation [6]. Modalities
146 here are not first-class type constructors and are rather properties of bindings, so types with modalities
147 cannot, for example, instantiate type variables, which limits the expressiveness further.

148 Contemporary research in effect systems makes extensive use of type-based escape analysis to formally guarantee safety properties. Xie et al. [10] adopt higher-rank types to enforce the non-leakage of
149 effect handlers. Capability-based effect systems employ type-based escape analysis to establish safety of
150 contextual polymorphism [8, 5, 4]. Recent work in the Scala language utilizes dependent types to check
151 escaping [3].

155 6 Conclusion

156 In this work we proposed the type-based escape analysis technique that trades some flexibility for simplicity of user-defined annotations. Namely, it includes minimum operation on lifetimes and simplifies
157 work with compositional data structures by approximating their lifetime as the minimum of lifetimes of
158 components. Also, we provided formalization and type-checker prototype implementation.

159 In the future work we will prepare mechanized soundness proof and evaluate trade-offs made by our
160 system using larger scale code bases.

162 References

- 163 [1] Marat Akhin and Mikhail Belyaev. Kotlin language specification. *Kotlin Language Specification*,
164 2021.
- 165 [2] Bruno Blanchet. Escape analysis for object-oriented languages: application to java. *Acm Sigplan*
166 *Notices*, 34(10):20–34, 1999.
- 167 [3] Aleksander Boruch-Gruszecki, Martin Odersky, Edward Lee, Ondřej Lhoták, and Jonathan
168 Brachthäuser. Capturing types. *ACM Transactions on Programming Languages and Systems*,
169 45(4):1–52, 2023.
- 170 [4] Jonathan Immanuel Brachthäuser, Philipp Schuster, Edward Lee, and Aleksander Boruch-Gruszecki.
171 Effects, capabilities, and boxes: from scope-based reasoning to type-based reasoning and back. *Proceedings of the ACM on Programming Languages*, 6(OOPSLA1):1–30, 2022.

- 173 [5] Jonathan Immanuel Brachthäuser, Philipp Schuster, and Klaus Ostermann. Effects as capabilities:
174 effect handlers and lightweight effect polymorphism. *Proceedings of the ACM on Programming Lan-*
175 *guages*, 4(OOPSLA):1–30, 2020.
- 176 [6] Anton Lorenzen, Leo White, Stephen Dolan, Richard A Eisenberg, and Sam Lindley. Oxidizing
177 ocaml with modal memory management. *Proceedings of the ACM on Programming Languages*,
178 8(ICFP):485–514, 2024.
- 179 [7] Nicholas D Matsakis and Felix S Klock. The rust language. In *Proceedings of the 2014 ACM SIGAda*
180 *annual conference on High integrity language technology*, pages 103–104, 2014.
- 181 [8] Leo Osvald, Grégory Essertel, Xilun Wu, Lilliam I González Alayón, and Tiark Rompf. Gentrification
182 gone too far? affordable 2nd-class values for fun and (co-) effect. *ACM SIGPLAN Notices*, 51(10):234–
183 251, 2016.
- 184 [9] Ross Tate. Mixed-site variance. In *FOOL’13: Informal Proceedings of the 20th International Work-*
185 *shop on Foundations of Object-Oriented Languages (Indianapolis, IN, USA)*. [http://www.cs.cornell.](http://www.cs.cornell.edu/~ross/publications/mixedsite/(cited%20on%20p.%2059))
186 [edu/~ross/publications/mixedsite/\(cited on p. 59\)](http://www.cs.cornell.edu/~ross/publications/mixedsite/(cited%20on%20p.%2059)), 2013.
- 187 [10] Ningning Xie, Youyou Cong, Kazuki Ikemori, and Daan Leijen. First-class names for effect handlers.
188 *Proceedings of the ACM on Programming Languages*, 6(OOPSLA2):30–59, 2022.